TryHackMe

OWASP Top 10 — 2025

Room Write-Up

Room Completion: 100% ✓
Date: March 25, 2026
Target IP: `10.49.137.138`
Environment: TryHackMe VPN + AttackBox

VULNERABILITIES COVERED

A04	A05	A08
Cryptographic Failures	Injection	Software & Data Integrity Failures

Task 1: Introduction

This TryHackMe room introduces three critical categories from the OWASP Top 10 (2025) list. The room provides both theoretical knowledge and hands-on exploitation practice covering application behaviour and user input vulnerabilities.

Categories Covered

- A04: Cryptographic Failures
- A05: Injection
- A08: Software or Data Integrity Failures

A target machine was deployed at IP 10.49.137.138. The room was accessed via TryHackMe VPN and an AttackBox environment.

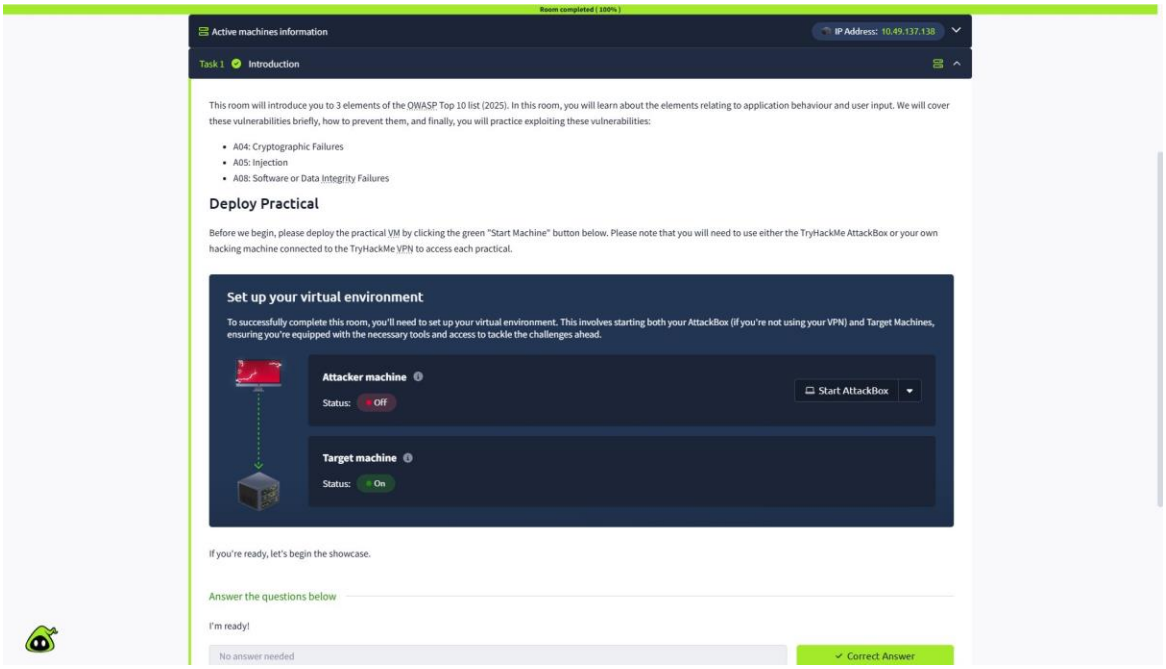

Figure 1: TryHackMe Room Introduction — OWASP Top 10 (2025) with target machine running

Task 2: A04 — Cryptographic Failures

What are Cryptographic Failures?

Cryptographic failures occur when sensitive data is not adequately protected due to lack of encryption, faulty implementation, or insufficient security measures.

Common Examples:

- Storing passwords without hashing
- Using weak or outdated algorithms such as MD5, SHA1, or DES
- Exposing encryption keys in source code or config files
- Failing to secure data in transit
- Rolling your own cryptography instead of using vetted algorithms

How to Prevent Cryptographic Failures

- Hash passwords using bcrypt, scrypt, or Argon2 — never MD5 or SHA1
- Never embed access credentials in source code or repositories
- Use secure key management systems for storing secrets
- Rely on trusted, industry-standard cryptographic libraries only

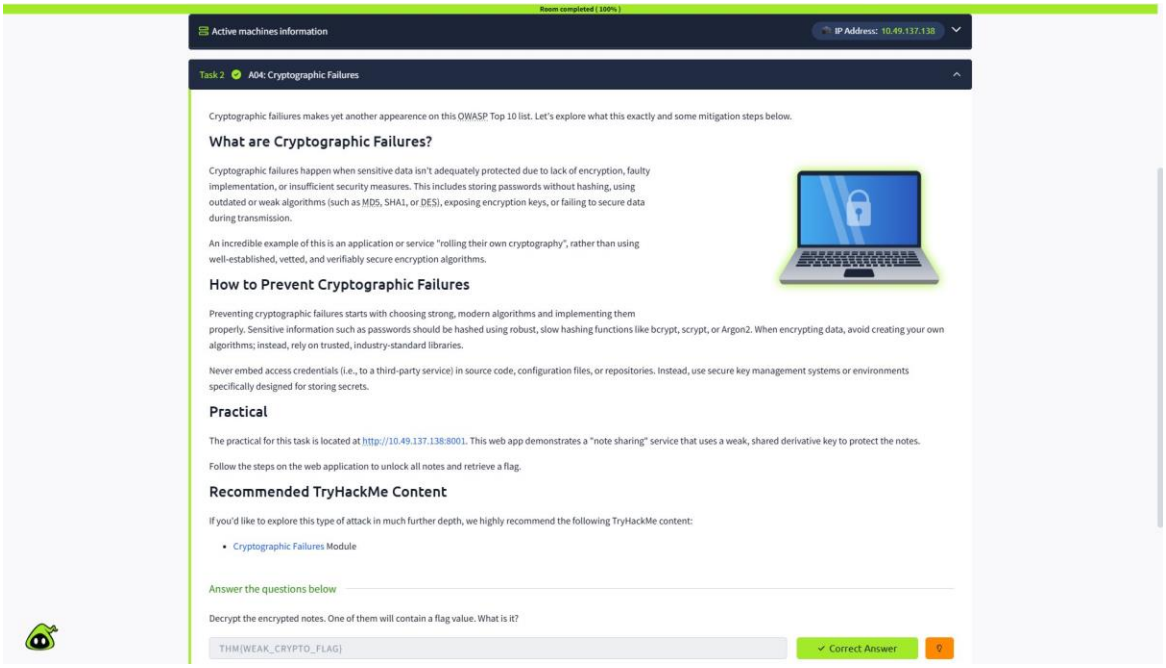

Figure 2: Task 2 — Cryptographic Failures overview and practical challenge

Practical — XOR Cipher Exploitation

The practical was hosted at http://10.49.137.138:8001. The application demonstrated a note-sharing service using a weak, shared derivative key.

Why This XOR Cipher is Insecure:

- Short keys (only 4 characters) can be brute-forced trivially
- Repeating key pattern enables frequency analysis
- No authentication — cannot verify if decryption succeeded
- Homegrown crypto lacks security guarantees of proven algorithms

Encrypted Notes (Base64-encoded):

```
Meeting Reminder #1:
BiA8RSIrPhE4JjFULzA1VC9lP145ZS1eJiorQyQyeVA/ZWoRGwh3EQgqN1cuNzxfKCB5QyQqNBEJaw==

Password Reset #2:
GyQqQjwqK1VrNzxCLjF5XSIrMgtrLS1FOzZjHmQsN0UuNzdQJ2stWSZqK1Q4IC00PyoyVCV4OFModGsC

Confidential #3:
GAAaYw4RYxEfDRRKHAAYehQGC2gbERZuDQkYdjZldBEPKnlfJDF5QiMkK1RrMTFYOGUuWD8teUQlJCxFIyorWDEgPRE7ICtCJC
```

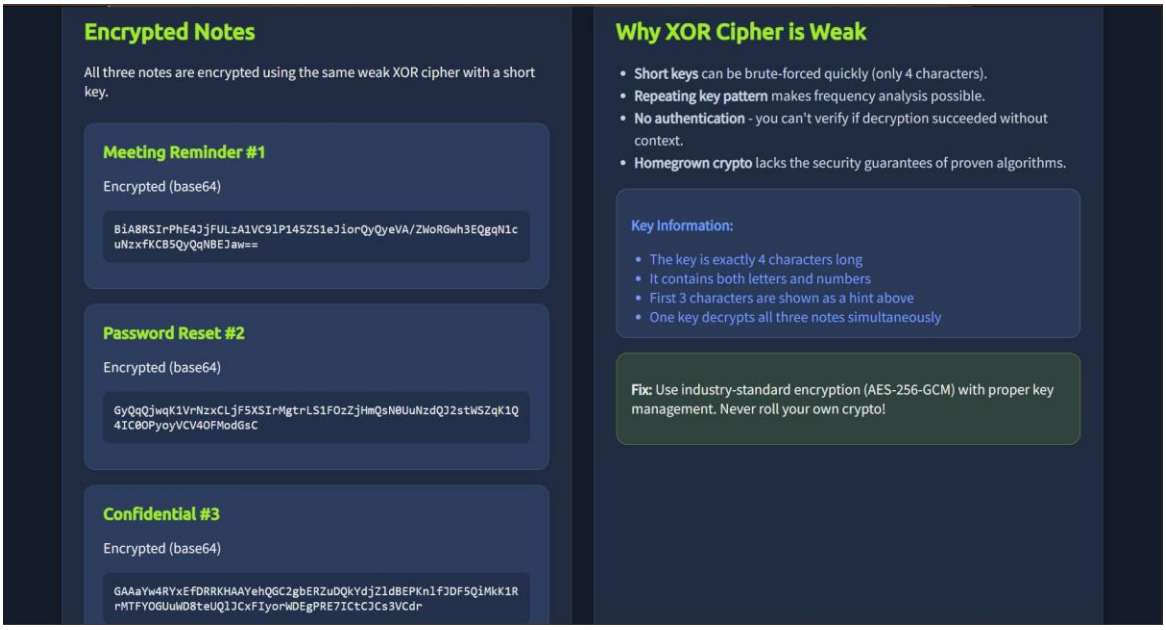

Figure 3: Encrypted notes and XOR cipher weaknesses shown on the target web app

Exploitation Steps:

The application hinted that the first 3 characters of the key are "KEY" and the 4th character is a digit.

Step 1 — Tested key "KEY1" in CyberChef using this recipe:

- From Base64 (A-Za-z0-9+/=, remove non-alphabet chars)
- XOR with key "KEY1" (UTF8, Standard scheme)

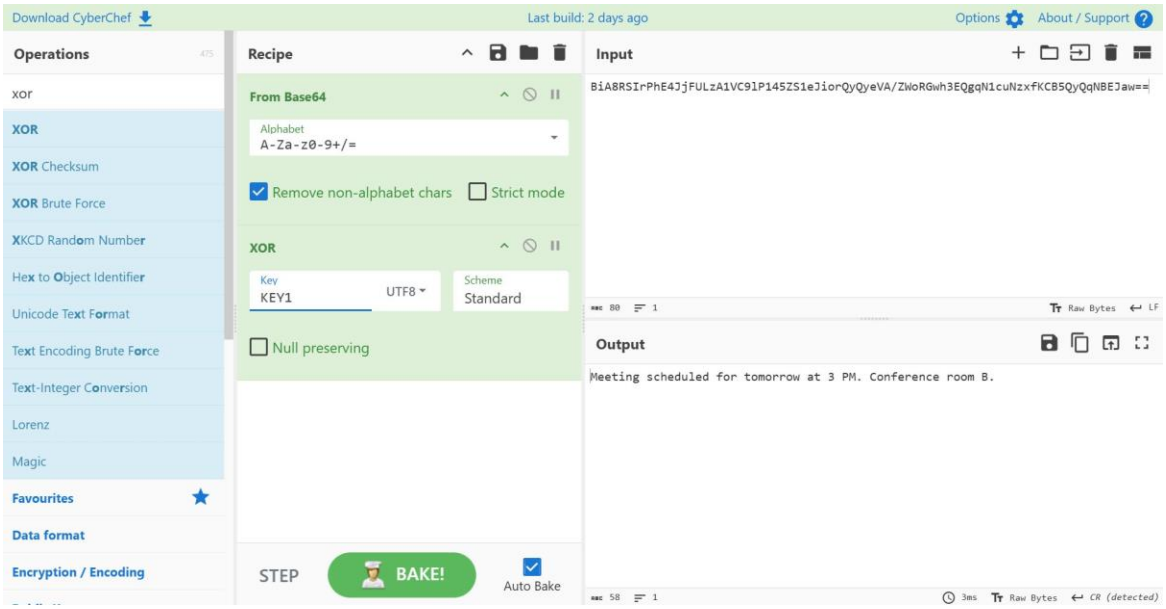

Figure 4: CyberChef decryption — From Base64 then XOR with key "KEY1" reveals plaintext

Step 2 — Entered key "KEY1" in the web application's "Try a Key" input box and clicked "Decrypt All Notes".

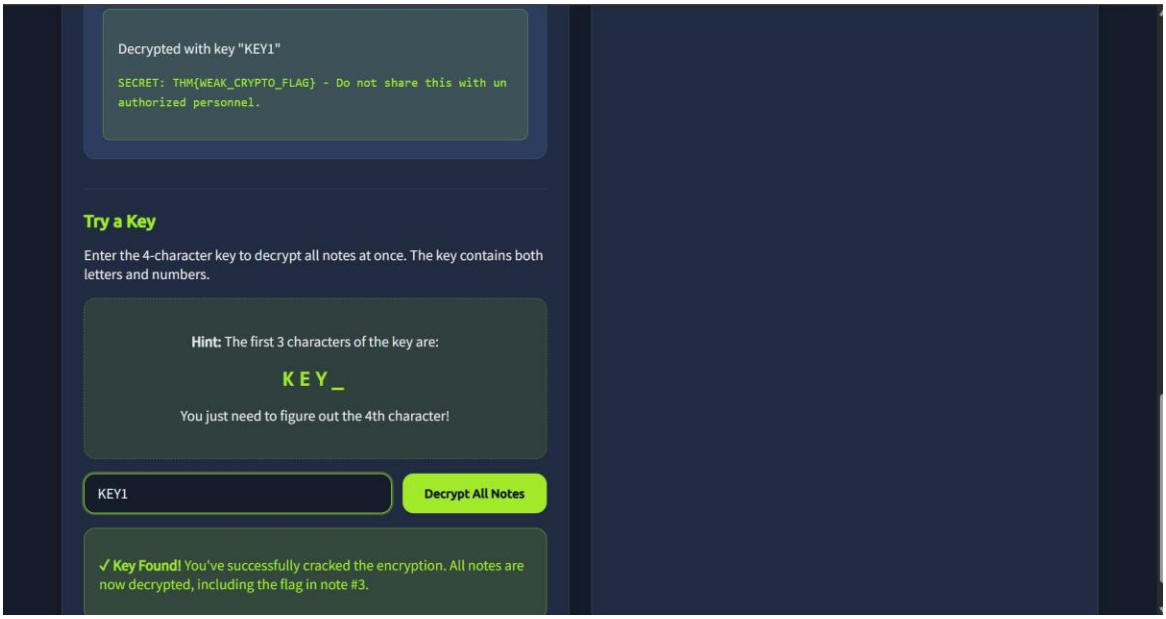

Figure 5: Web app confirms "Key Found!" — all notes decrypted with key KEY1, flag found in note #3

FLAG CAPTURED
THM{WEAK_CRYPTO_FLAG}

Task 3: A05 — Injection

What is Injection?

Injection occurs when an application takes user input and mishandles it — passing it directly into a system that can execute commands or queries, such as a database, shell, templating engine, or API.

Classic Examples of Injection Attacks:

- SQL Injection
- Command Injection
- AI Prompt Injection
- Server Side Template Injection (SSTI)

⚠ High Severity
Even in 2025, injection attacks remain highly relevant — appearing on the OWASP Top 10 list twice by 2025. This vulnerability class must be treated with the utmost priority.

How to Prevent Injection

- Use prepared statements and parameterised queries — never build SQL via string concatenation
- Avoid functions that pass input directly to the OS shell; use safe APIs
- Implement input validation and sanitisation — escape dangerous characters
- Enforce strict data types and filter inputs before the application processes them

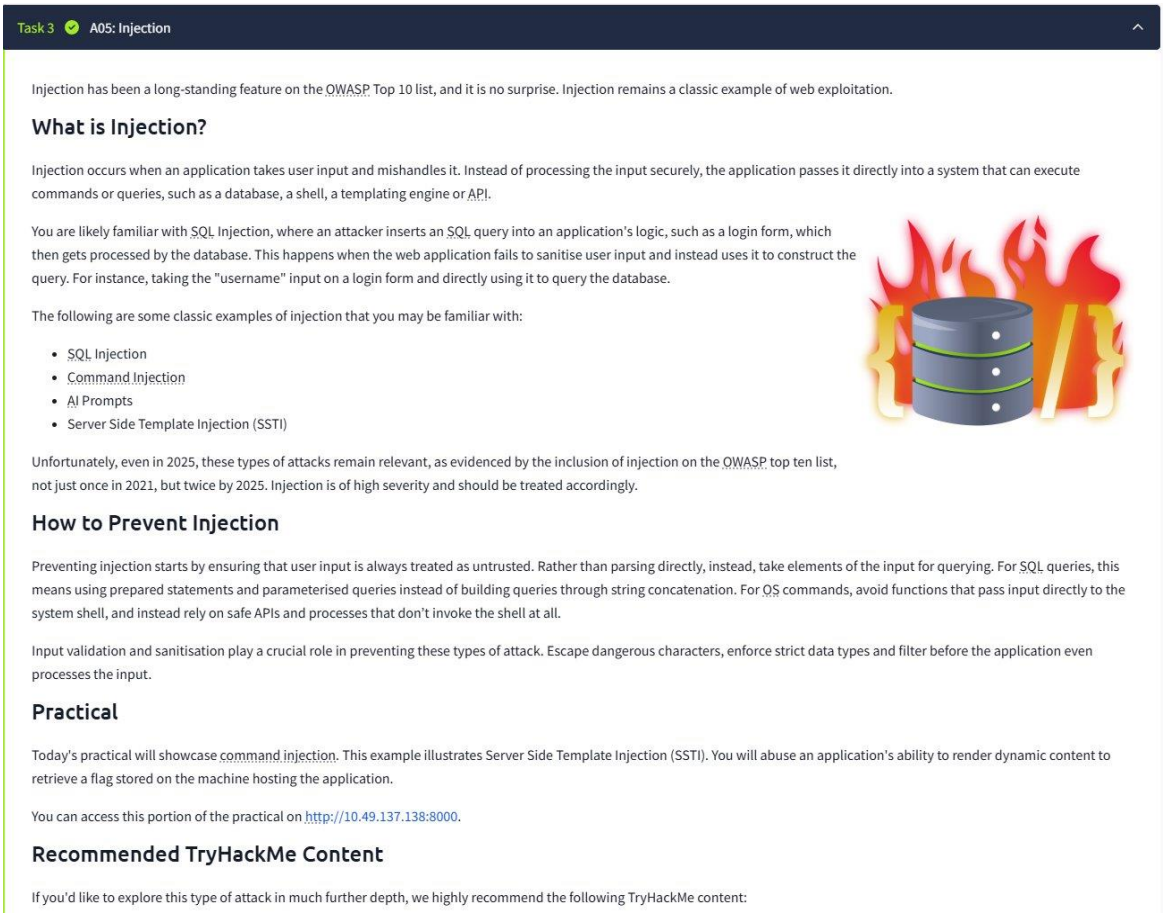

Figure 6: Task 3 — A05 Injection overview including prevention strategies

Practical — Server Side Template Injection (SSTI)

The practical demonstrated command injection via SSTI. The target was http://10.49.137.138:8000. The objective was to read flag.txt from the web application's directory.

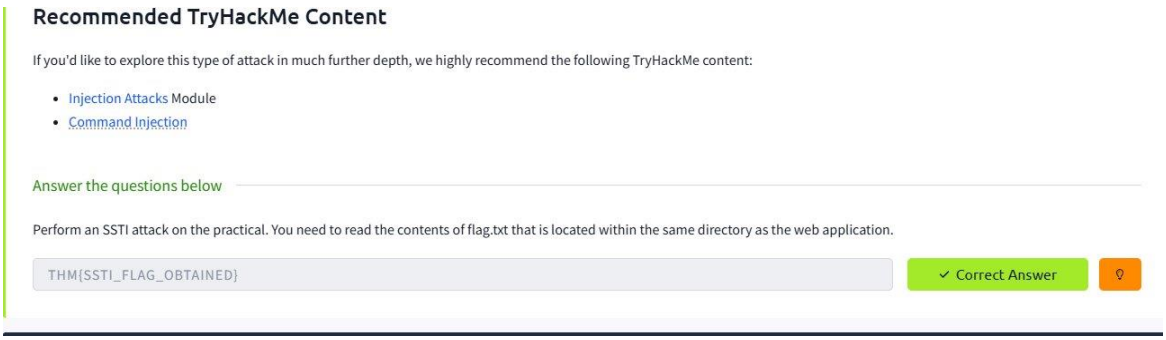

Figure 7: Task 3 question — SSTI attack to read flag.txt

The Vulnerability

The application used render_template_string to process raw user input inside a Jinja2 template — without any sandboxing. This allowed access to Python internals.

Attack Vectors Available:

- Jinja2 templates access Python objects when not sandboxed
- Built-ins like config, request, cycler, joiner, and lipsum are accessible
- Payloads like {{7*7}} or {{config.items()}} can probe the environment

Malicious Payload Used:

```
{{ self.__init__.__globals__.__builtins__.open("flag.txt").read() }}
```

User-controlled strings are evaluated as templates. Attackers can call Python builtins or leverage objects to access host resources. The rendered output returned the flag directly.

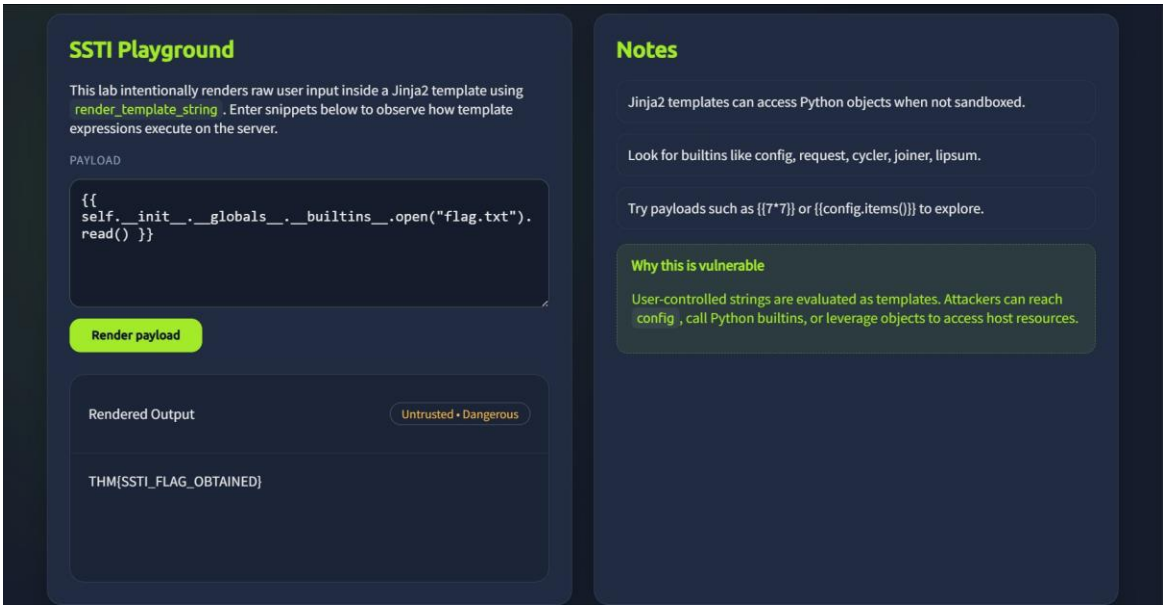

Figure 8: SSTI Playground — payload executed successfully, returning flag from flag.txt

> **FLAG CAPTURED**
> THM{SSTI_FLAG_OBTAINED}

Task 4: A08 — Software or Data Integrity Failures

What Are Software or Data Integrity Failures?

These failures occur when an application relies on code, updates, or data it assumes are safe — without verifying their authenticity, integrity, or origin.

This Includes:

- Trusting software updates without cryptographic verification
- Loading scripts or config files from untrusted sources
- Failing to validate data that impacts application logic
- Accepting binaries, templates, or JSON without confirming they have not been altered

How to Prevent These Failures

- Never assume code, updates, or key data are automatically trustworthy
- Use cryptographic checksums for update packages
- Ensure only trusted sources can modify critical artefacts
- Enforce integrity and trust boundaries in CI/CD build pipelines

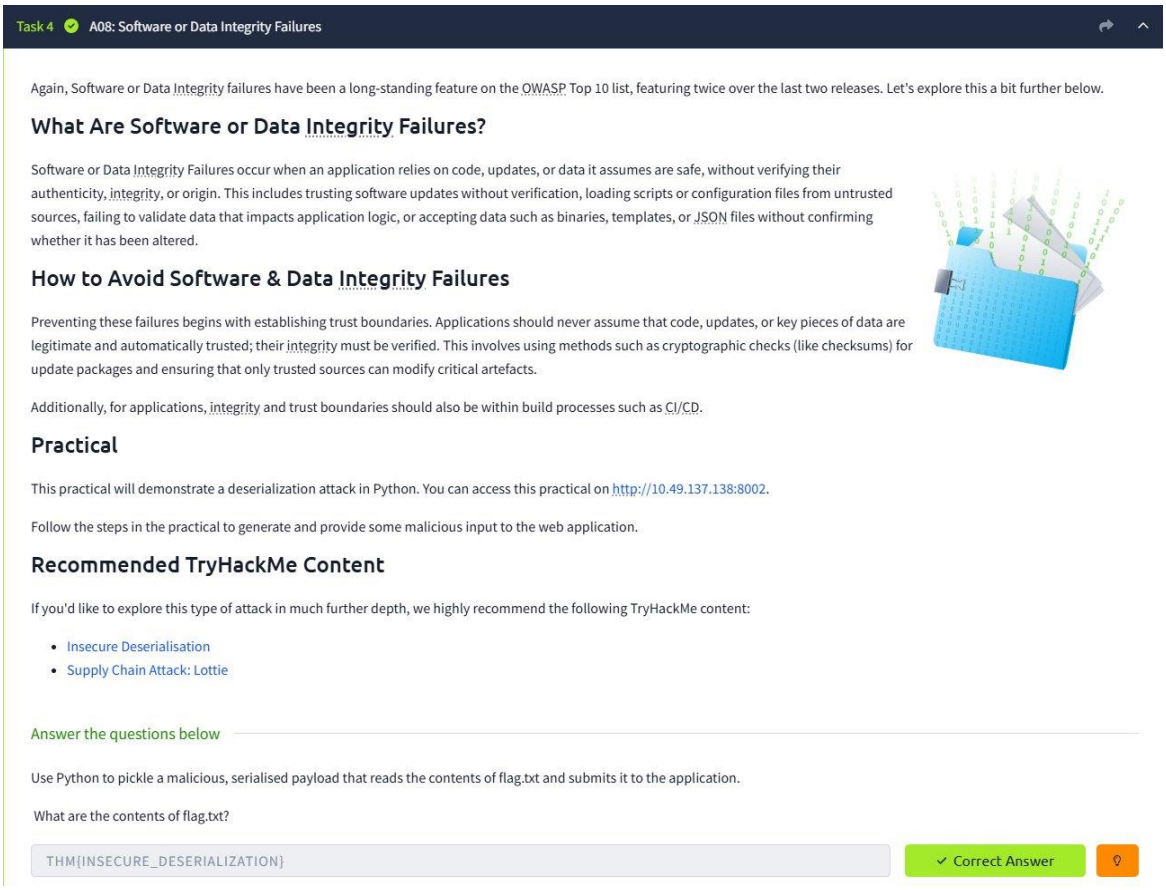

Figure 9: Task 4 — A08 Software or Data Integrity Failures overview and practical description

Practical — Insecure Deserialization (Python Pickle)

This practical demonstrated a deserialization attack. Target: http://10.49.137.138:8002. The objective was to generate a malicious serialized payload that reads flag.txt and submit it.

Why This Application is Vulnerable:

- No integrity verification — accepts any pickle data without checking signatures or hashes
- Unsafe deserialization — Python's pickle can execute arbitrary code during deserialization
- Trust boundary violation — untrusted user input is deserialized as if it were trusted
- No input validation — no schema validation or whitelisting of allowed object types

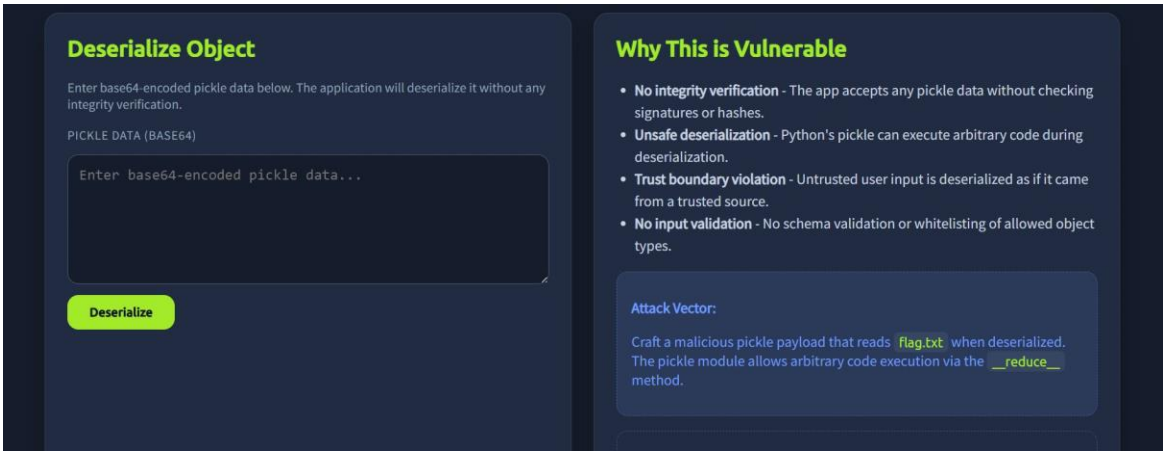

Figure 10: Target application — Deserialize Object form and vulnerability explanation

Crafting the Malicious Payload

A Python script was written using the __reduce__ method to achieve arbitrary code execution:

```
import pickle
import base64

class Malicious:
    def __reduce__(self):
        # Return a tuple: (callable, args)
        # This will execute: open('flag.txt').read()
        return (eval, ("open('flag.txt').read()",))
```

```
# Generate and encode the payload
payload = pickle.dumps(Malicious())
encoded = base64.b64encode(payload).decode()
print(encoded)
```

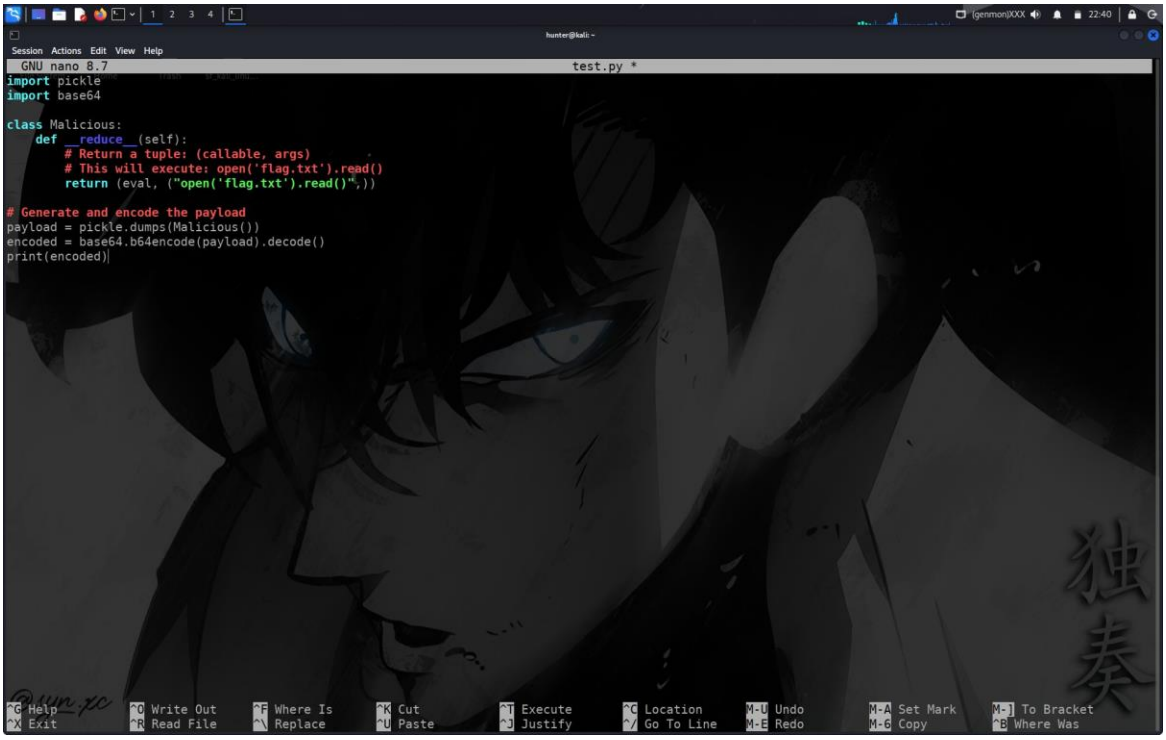

Figure 11: Kali Linux terminal — test.py written in nano to generate the malicious pickle payload

Executing and Submitting the Payload

The script was executed using Python 3, producing a base64-encoded malicious payload:

```
$ python3 test.py
gASVMwAAAAACMCGJ1aWx0aW5zAJSTlIwXb3BlignZmxhZy50eHQeHQnKS5yZWFkKCmUhZRslC4=
```

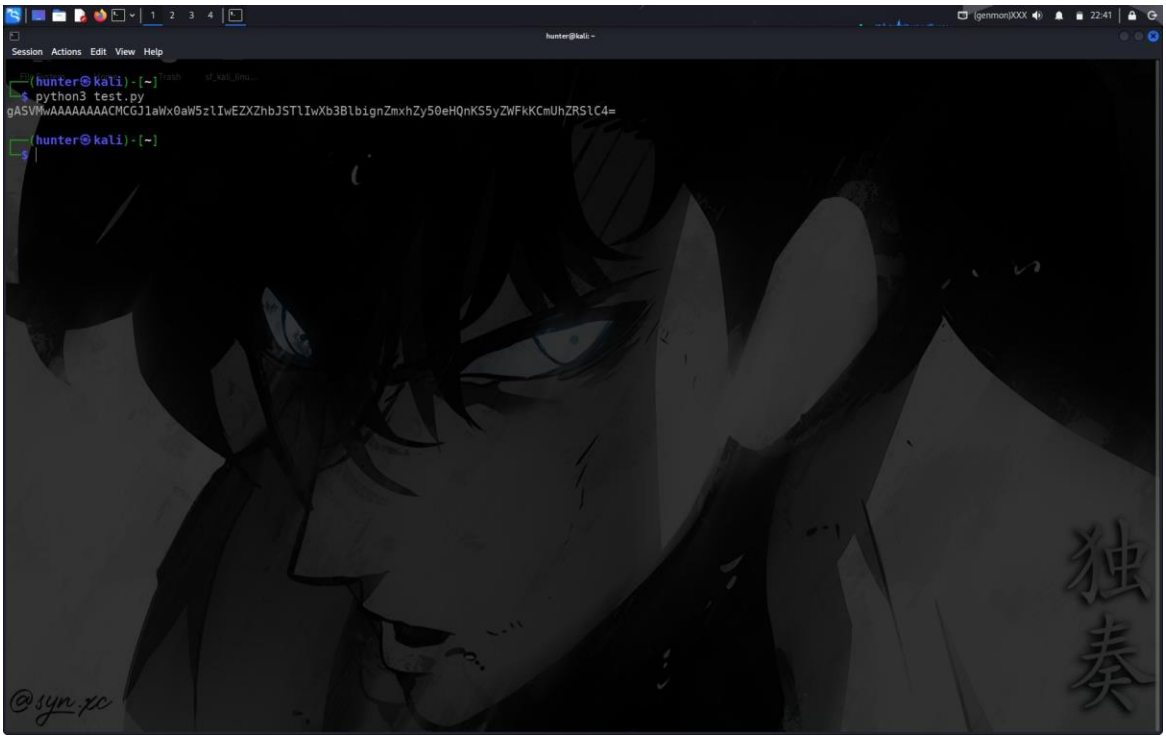

Figure 12: Terminal output — base64-encoded malicious pickle payload generated successfully

The generated base64 payload was then submitted to the web application's Deserialize form:

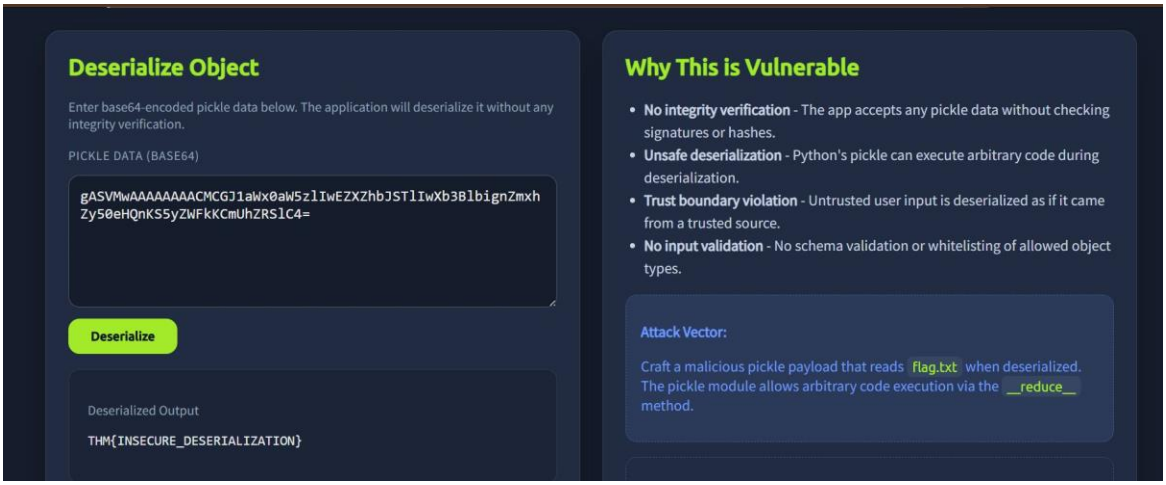

Figure 13: Payload submitted — deserialized output reveals flag.txt contents

> **FLAG CAPTURED**
> THM{INSECURE_DESERIALIZATION}

Summary & Key Takeaways

All tasks in this TryHackMe room were completed successfully, achieving 100% room completion. Below is a consolidated summary of the three OWASP vulnerabilities exploited.

Results Overview

OWASP Category	Attack Type	Flag Captured	Status
A04: Cryptographic Failures	Weak XOR Cipher Brute Force	`THM{WEAK_CRYPTO_FLAG}`	✓ Solved
A05: Injection	SSTI via Jinja2	`THM{SSTI_FLAG_OBTAINED}`	✓ Solved
A08: Data Integrity Failures	Python Pickle Deserialization	`THM{INSECURE_DESERIALIZATION}`	✓ Solved

Lessons Learned

- **Never use homegrown or weak cryptographic implementations — always use AES-256-GCM or similar industry-standard algorithms**
- Never roll your own crypto; rely on vetted, well-established libraries
- **Sanitise and validate all user input before it reaches template engines, databases, or OS commands**
- For Python web apps using Jinja2, never use render_template_string with unsanitised user input
- **Never deserialise untrusted data with Python's pickle module — use safe formats like JSON with schema validation**
- Always verify the integrity and authenticity of data using cryptographic signatures or checksums

> **Room Complete**
> All three OWASP Top 10 vulnerabilities were successfully identified, understood, and exploited. This room demonstrates real-world attack techniques and reinforces defensive security best practices.